MAPPET

Plano de Segurança

Incremento atual: UC01 a UC11
Java + Spring Boot + Oracle
Outubro de 2026

Este plano mostra o que o MAPPET precisa proteger e como cada regra de segurança prevista nos casos de uso, no MER e no MLR será garantida no back end em Java com Spring Boot.

1 O que precisamos proteger

Quadro 1: Dados que exigem cuidado

Dado	Onde fica (MLR)	Por que proteger
Senha	`T_MAPPET_USUARIO.SENHA_HASH`	Muita gente usa a mesma senha em outros sites
E-mail e telefone	`T_MAPPET_USUARIO.EMAIL`, `TELEFONE`	São dados pessoais de contato; o do acolhedor só aparece pelo UC11
Nome	`T_MAPPET_USUARIO.NOME`	Identifica a pessoa
Fotos	arquivo apontado por `FOTO_URL`	Podem mostrar a casa ou o rosto de quem publicou
Registro de interesse	`T_MAPPET_INTERESSE_ADOCAO`	Mostra quem recebeu o contato de cada acolhedor e quando; é o que permite rastrear um caso de abandono ou maus-tratos depois da adoção

Fonte: elaborado pelas autoras (2026), a partir do MLR.

2 Regras dos documentos e como o sistema garante cada uma

Quadro 2: Regras de segurança previstas nos documentos de referência

Regra	Origem	Como o sistema garante
Senha armazenada de forma criptografada, nunca em texto puro	UC01 passo 7 RF01, RNF02; MLR	O back end grava apenas o hash da senha, gerado com o `BCryptPasswordEncoder` do Spring Security, na coluna `SENHA_HASH`
Um e-mail só pode ter uma conta	UC01 E2; MLR	Coluna `EMAIL` com `UNIQUE`; a API avisa que o e-mail já existe e sugere o login
Login por e-mail e senha, informando que "e-mail ou senha estão incorretos"	UC02 E2 RF07, RN01	A mensagem é a mesma para os dois casos, sem dizer qual dos dois está errado
Usuário pode sair da conta a qualquer momento	UC02 A2	A rota de logout invalida a `HttpSession` no servidor
Home, publicações e área de adoção só para quem fez login	UC02, UC03, UC05, UC06, UC09, UC10, UC11	O `LoginInterceptor` bloqueia essas rotas e responde 401 sem login
Na página de login, só o botão "Adote aqui" e alguns animais em destaque	UC09 RN07	`GET /adocao/vitrine` é a única rota de adoção aberta e devolve só foto, nome e espécie; a lista completa (`GET /adocao`) exige login

Regra	Origem	Como o sistema garante
Contato do acolhedor só depois de registrar o interesse	UC11 RN07	`POST /adocao/{id}/interesse` grava adotante, animal e data em `T_MAPPET_INTERESSE_ADOCAO` e só então devolve o contato
Um registro de interesse por adotante e animal	UC11 A2; MLR	Chave primária composta (`ID_USUARIO`, `ID_ANIMAL`); se o interesse já existe, a API só mostra o contato de novo
Toda publicação, todo animal e todo interesse pertencem a uma conta	UC05, UC06, UC10, UC11 RF02, RF03; MLR	`ID_USUARIO NOT NULL`, preenchido com o usuário da sessão, nunca com um valor vindo do formulário
Foto só em JPG ou PNG, até 5 MB	UC07 E1 RNF06, RNF08; MLR	O `MultipartFile` é conferido pelo tipo e pelos primeiros bytes do arquivo, com limite de 5 MB no `application.properties`; o arquivo é salvo com nome gerado pelo sistema
Nome, espécie, bairro e foto obrigatórios; nome e bairro sem números e não só espaços; espécie da lista	UC08 RN02; MLR	Anotações do Bean Validation (`@NotBlank`, `@Pattern`, `@Size`) com `@Valid`; o back end valida de novo tudo o que o site já validou
Cadastro: nome sem números, e-mail válido, telefone só com números	UC01 passo 5, E1	Mesma validação no site e no back end (`@Email`, `@Pattern`)
Cards da Home mostram foto, nome, espécie, bairro e tipo	UC03 RF04	A resposta da Home usa um DTO só com esses campos; e-mail e telefone de quem publicou não são enviados

Fonte: elaborado pelas autoras (2026), a partir dos casos de uso, do MER e do MLR.

3 Cuidados técnicos da implementação

Estes cuidados não mudam nenhuma regra dos documentos. São a forma correta de programar o que já está previsto:

- **Consultas com parâmetros**, para evitar SQL injection (alguém digitar um comando SQL num campo do formulário). Com Spring Data JPA, os métodos do repository já usam parâmetros:

```
// errado: o texto digitado vira parte do SQL
String sql = "SELECT * FROM T_MAPPET_USUARIO WHERE EMAIL = '" + email + "'";

// certo: o Spring Data JPA monta a consulta com parâmetro
Optional<Usuario> findByEmail(String email);
```

- **Senhas do sistema fora do GitHub.** O `application.properties` lê a senha do banco de uma variável de ambiente (`${DB_PASSWORD}`), sem escrever a senha real no arquivo.
- **Respostas sem dados demais.** As rotas devolvem DTOs, nunca a entidade inteira, para o `SENHA_HASH` e os contatos não vazarem por engano.
- **HTTPS** quando o site for publicado na internet, para que senha e dados não trafeguem abertos.
- **Dados fictícios** nos testes, nunca dados de pessoas reais.

4 Checklist antes da entrega

- [ ] Senhas no banco aparecem como hash (começam com `$2a$`), nunca legíveis (UC01)
- [ ] Cadastrar duas vezes o mesmo e-mail é recusado (UC01 E2)
- [ ] Senha errada e e-mail inexistente mostram a mesma mensagem (UC02 E2)
- [ ] Abrir a Home, publicar ou abrir a área de adoção sem login devolve erro 401 (UC03, UC05, UC06, UC09, UC10)
- [ ] Sem login, a vitrine mostra só foto, nome e espécie (UC09)
- [ ] O contato do acolhedor só aparece depois do "Quero adotar", e o interesse fica gravado (UC11)
- [ ] Clicar de novo em "Quero adotar" no mesmo animal não cria outro registro (UC11 A2)
- [ ] Arquivo que não é JPG ou PNG, ou maior que 5 MB, é recusado (UC07 E1)
- [ ] Campos vazios, só com espaços ou com números são recusados pelo back end (UC08)
- [ ] Nenhuma consulta SQL junta texto digitado pelo usuário
- [ ] A senha do banco não está escrita no código nem no GitHub

Referências: casos de uso UC01 a UC11, MER e MLR do MAPPET; Lei nº 13.709/2018 (LGPD).